

Fibers and `yield`

How the interpreter suspends a program, and how to drive one that has

Contents

1	What this describes	2
1.1	The shape of the thing, in one page	2
2	The four types	3
2.1	Frame: one block with work left in it	3
2.2	FiberState: the program	3
3	The one restriction	5
3.1	Why the restriction exists	5
3.2	Where the line falls	5
4	Permission: how the restriction is enforced	7
4.1	The three rules	7
4.2	Worked examples of the rule	8
5	Building the record: every step	9
5.1	The signal	9
5.2	Step by step	9
5.3	The ordering invariant	11
6	Jumping back in	12
6.1	Innermost first, one frame at a time	12
6.2	The outer frames are carried forward	12
6.3	The value is carried outward	12
7	Interaction with everything else	14
7.1	Errors and throws	14
7.2	Fuel	14
7.3	Threads	14
8	Writing an event loop	15
8.1	The minimum loop	15
8.2	What the yielded value is for	15
8.3	Worked example: a cooperative scheduler	15
8.4	Worked example: this interpreter's editor	16
8.5	What a fiber may not do	17
9	Reference	18
9.1	The Lisp surface	18
9.2	The Rust surface	18
9.3	Invariants worth not breaking	18
9.4	Where the code is	18

1 What this describes

The interpreter can stop a running program in the middle of itself, hand a value to whoever started it, and later carry on from the exact place it stopped — with its variables, its loops and its half-finished function calls all as they were. A program that can do that is a *fiber* the act of stopping is `yield`; the act of carrying on is `resume`.

This document is about the machinery underneath. It says, in order:

- what a fiber is made of, and why it is made of exactly that;
- the one restriction on where a program may stop, and the reason it cannot be lifted without rewriting the evaluator;
- every step the interpreter takes to build the record of where a program stopped;
- every place that record can be entered again;
- how to write an event loop that drives fibers.

The interpreter knows nothing about editors, terminals or files. Everything here is true of any host that embeds it. Where an example needs a setting, the editor this interpreter ships inside is used — but nothing in the mechanism depends on it, and a host that scheduled fibers against a network socket or a game frame would use the same three calls.

1.1 The shape of the thing, in one page

A fiber is a *stack of blocks that still have work left in them*.

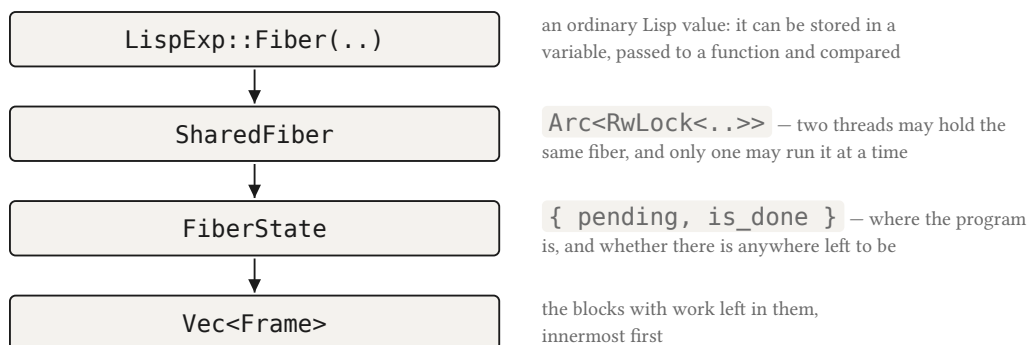
```
(setq counter
  (fiber
    (let ((n 0))
      (while t
        (setq n (+ n 1))
        (yield n))))))

(resume counter) ; => 1
(resume counter) ; => 2
(resume counter) ; => 3
```

The first `resume` runs until it reaches the `yield`, hands back `1`, and stops. What it leaves behind is not a saved copy of the machine — it is a short list saying “**you were part-way through the body of this while, at index 2, in this environment**”. The second `resume` reads that list and carries on. Everything else in this document is the detail of building that list and of reading it back.

2 The four types

Four types stand between a Lisp value and a suspended program. Each one exists because the one below it could not answer a question.



2.1 Frame: one block with work left in it

```
pub enum Frame<T: LispContext> {
    Body { forms: Vec<LispExp<T>>, from: usize, env: Arc<Env<T>> },
    While { condition: LispExp<T>, forms: Vec<LispExp<T>>,
           from: usize, env: Arc<Env<T>> },
}
```

A frame is a sentence: *run these forms, starting at this index, in this environment*. `While` adds one clause – and then test the condition and go round again.

Three fields, and each is load-bearing:

forms the block’s own source, kept whole rather than truncated. Keeping the whole body means a `While` frame can go back to index 0 on the next iteration; a frame holding only “what is left” could not.

from where to pick up. Everything before it has already run and must not run again – this is the field that a resumption which restarted the block would get wrong, and a loop that ran its first half twice per turn is the symptom.

env the scope the block was running in. The code alone is not enough: a `let` body resumed in its parent’s environment would be the right statements reading the wrong variables, and it would look correct in every test whose names happened not to collide.

2.2 FiberState: the program

```
pub struct FiberState<T: LispContext> {
    pub pending: Vec<Frame<T>>,
    pub is_done: bool,
}
```

That is the whole of a coroutine’s state. `pending` is where it is; `is_done` is whether there is anywhere left to be.

A fiber that has never been started is a fiber suspended *before its first form*, which is a position like any other. So `(fiber A B C)` builds `pending = [Body { forms: [A, B, C], from: 0, env }]` and nothing else. There is no separate “not started yet” state, and `resume` therefore has one path rather than two.

This is worth dwelling on, because the alternative was in the tree for a while. `FiberState` used to carry the top-level forms and the root environment *beside* the stack of frames — and “the forms of this body from index 0, in this environment” is precisely what a `Body` frame already says. Two representations of one fact meant `resume` had to ask which it was looking at, and the answer was an enum with three variants that existed only to be matched on once.

Folding the two together changed one visible behaviour. A fiber’s body used to run *one top-level form per resume*. It now runs to the end unless a `yield` stops it, exactly as a `progn` does. The old stepping was a second, invisible way of suspending — a body of three forms suspended twice with no `yield` anywhere in the source — and it stopped earning its confusion the moment a real `yield` existed. Write the yields you mean:

```
(fiber A (yield) B (yield) C) ; three resumes, as before
(fiber A B C)                 ; one resume, all three forms
```

3 The one restriction

A fiber may only stop where the value of what it is running is going to be thrown away.

`(yield)` is legal in **statement position**: a form in a body whose value is discarded, or a form in a loop body. It is refused anywhere else.

3.1 Why the restriction exists

The evaluator is an ordinary recursive Rust function. When it is part-way through `(+ 1 (f x))`, the fact that there is a `+` waiting for a second operand lives in a Rust stack frame — in the `evaluated_args` vector of `eval_function_call_step`, on the machine stack, with a return address. Suspending means leaving that Rust frame. Resuming would mean recreating it and delivering a value into the middle of it.

Nothing short of rewriting the evaluator as an explicit state machine, with its own heap-allocated continuation stack, can do that. The two standard ways are a full CPS transform of every evaluation rule, or a bytecode VM with an operand stack that can be saved. Both are large, and both change the cost of *every* evaluation to buy something that matters in a handful of places.

In statement position the problem disappears, because there is nothing to deliver. The value is discarded either way, so coming back means only “**run the rest of these forms**” — and that is a list of expressions plus an index, which is exactly `Frame`.

This is the same boundary Lua draws around its coroutines, and for the same reason: it is the line between “the continuation is data I already have” and “the continuation is the C stack”.

3.2 Where the line falls

Position	Example	yield?
Body statement	<code>(progn (yield) x)</code>	yes
Loop body	<code>(while c (yield))</code>	yes
<code>let</code> / <code>let*</code> body	<code>(let ((a 1)) (yield) a)</code>	yes
<code>when</code> / <code>unless</code> body	<code>(when c (yield))</code>	yes
<code>cond</code> clause body	<code>(cond (c (yield) 1))</code>	yes
Function body	<code>(defun f () (yield) 1)</code>	yes
Tail position of any of these	<code>(when c (yield))</code>	yes
Argument	<code>(+ 1 (yield))</code>	no
Condition	<code>(if (yield) a b)</code>	no
<code>let</code> binding value	<code>(let ((a (yield))) a)</code>	no
<code>and</code> / <code>or</code> operand	<code>(and (yield) t)</code>	no
<code>dolist</code> / <code>dotimes</code> body	<code>(dolist (x l) (yield))</code>	no
<code>unwind-protect</code> body	<code>(unwind-protect (yield) c)</code>	no
<code>condition-case</code> body	<code>(condition-case e (yield) ..)</code>	no
<code>catch</code> body	<code>(catch 'tag (yield))</code>	no
Inside a primitive’s callback	<code>(mapcar (lambda (x) (yield)) l)</code>	no
Outside a fiber altogether	<code>(yield)</code> at top level	no

A refused `yield` raises `EvalError::YieldNotAllowed`, and raises it *before* evaluating its own argument — so a program that yields somewhere impossible does not get half of its side effects done first.

Three of the refusals are worth a word each. `dolist` and `dotimes` keep their position in Rust rather than in a frame, so there is nothing to come back to; a fiber that wants to iterate across turns writes a `while`. `unwind-protect` would have to choose between running its cleanup for a form that has not finished and skipping it for one that may never resume, and refusing is the honest third answer. `condition-case` and `catch` install a handler on a Rust stack that suspension destroys; a fiber resumed later would be running inside a handler that no longer exists.

4 Permission: how the restriction is enforced

The restriction is not a list of checks. It is enforced by a single thread-local flag that the recordable positions *grant* and every evaluation *consumes*.

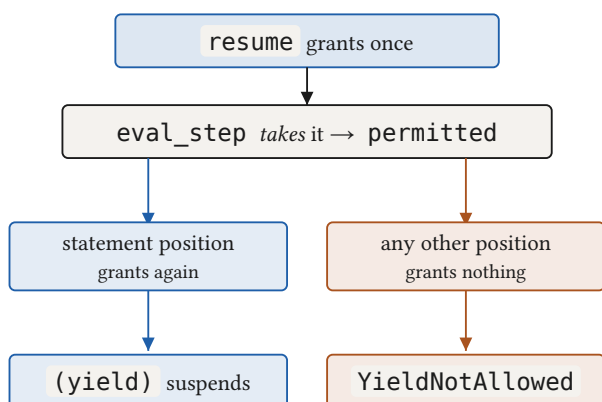
```
thread_local! {
    static YIELD_PERMITTED: Cell<bool> = const { Cell::new(false) };
}

pub fn grant_yield_permission() { YIELD_PERMITTED.set(true); }
fn take_yield_permission() -> bool { YIELD_PERMITTED.replace(false) }
```

4.1 The three rules

1. **Default false.** A position that has never been taught to record a frame has no permission to give away. Adding a new special form cannot accidentally allow a suspension that would then be lost — the worst it can do is refuse one that could in principle have been supported.
2. **Consumed on entry.** `eval_step` takes the flag the moment an evaluation begins, leaving none for anything nested inside it. Without this, `(progn (foo (yield)))` would let the argument inherit the statement's permission and suspend with no record of the half-built call to `foo`.
3. **Re-granted on a tail call.** A form tail-called from a body has nothing of that body left after it, so whatever could have recorded for the body can record for it. `eval_step` puts the permission back when a step turns out to be a tail call, which is what makes `(when ready (yield))` — a fiber's ordinary way of saying “nothing to do just now” — work.

```
fn eval_step<T: LispContext>(exp: &LispExp<T>, env: Arc<Env<T>>, ctx: &T)
    -> Result<EvalStep<T>, EvalError<T>>
{
    let permitted = take_yield_permission();           // rule 2
    let step = eval_step_permitted(exp, env, ctx, permitted);
    if permitted && matches!(step, Ok(EvalStep::TailCall(..))) {
        grant_yield_permission();                     // rule 3
    }
    step
}
```



The flag is **taken**, not read.
Everything below an evaluation starts from nothing, so only the positions that can record a frame hand permission downwards.

4.2 Worked examples of the rule

```
(fiber (while t (do-a) (yield) (do-b)))
```

`resume` grants; `eval_step` consumes it for the `while` form; the `while` arm sees `permitted` and grants again before each body form; `(yield)` consumes that grant and suspends. *Legal*.

```
(fiber (foo (yield)))
```

`resume` grants; `eval_step` consumes it for `(foo (yield))`; the call evaluates its arguments with permission already gone; `(yield)` sees `false`. *Refused*, and no record of the half-built call to `foo` was ever made.

```
(fiber (progn (unwind-protect (yield) (cleanup))))
```

`progn` grants for the `unwind-protect` statement; `eval_step` consumes it; `unwind-protect` evaluates its body with nothing granted. *Refused* — and note that no check for `unwind-protect` appears anywhere. It is refused because it was never taught to grant.

This is the property worth keeping when the evaluator is extended: a suspension can never escape a construct that cannot record it, because the permission that makes a suspension legal is never granted there. It is correctness by construction rather than by a list somebody has to maintain.

5 Building the record: every step

This is the heart of it. A `(yield)` deep inside a program has to leave behind a list of frames describing the whole way back out, and it does that by unwinding — each block that *can* say where it was, says so on the way past.

5.1 The signal

```
pub enum EvalError<T: LispContext> {  
    // ...  
    Yielded { value: LispExp<T>, frames: Vec<Frame<T>> },  
    YieldNotAllowed,  
}
```

A suspension travels on the error channel. That is not a claim that it is an error — it is not — and the reason is mechanical, and worth stating plainly because it is the first thing a reader questions.

`eval` has the signature `Result<LispExp<T>, EvalError<T>>`. The error half of a `Result` is the only channel that `?` propagates automatically. Putting a suspension anywhere else means every one of the hundred-odd places that evaluate a sub-expression — every special form, every primitive that calls back into Lisp — has to test for it and hand it on by hand, and the failure mode of forgetting one is that a suspension silently becomes an ordinary value.

The alternative — a `LispExp::Suspended` variant — is not slower. Both are plain tagged returns; there is no unwinding in the C++ or panic sense, and no allocation either. `EvalError` is 56 bytes with `Yielded` and 56 bytes without, because `Throw { tag, value }` already sets that floor; the variant is free. The whole of the difference is that one design propagates by construction and the other propagates by discipline, and the discipline would have to be exercised by every primitive anyone ever adds.

`Throw` sits on the same channel and has always done so, for the same reason: a non-local exit is a thing that leaves the middle of a computation, and leaving the middle of a computation in a recursive evaluator means returning `Err` all the way up. `EvalError` is best read as “**the ways an evaluation can end other than with a value**” — three of which are errors, and two of which are not.

5.2 Step by step

Take a concrete program and follow it.

```
(fiber  
  (let ((n 0))  
    (while (< n 3)  
      (log "top")  
      (yield n)  
      (setq n (+ n 1))))))
```

5.2.1 Step 0 — the fiber is built

`(fiber ...)` evaluates to a `LispExp::Fiber` whose state is:

```
pending = [ Body { forms: [(let ..)], from: 0, env: <child of caller> } ]  
is_done = false
```

No evaluation has happened. The body is stored, not run.

5.2.2 Step 1 — *resume takes the stack and grants permission*

`primitive_resume` locks the fiber, takes `pending` out of it with `mem::take`, and drops the lock before evaluating anything. Two things follow from taking rather than borrowing:

- Lisp that reaches back to its own fiber — asking whether it has finished, or resuming it — cannot deadlock on a lock nobody can see.
- A second `resume` arriving from another thread finds an empty stack and does nothing, rather than starting the same program a second time alongside the first.

5.2.3 Step 2 — *resume_frames runs the bottom frame*

With one frame on the stack, `resume_frames` calls `run_statements` on it. `run_statements` is the single definition of “statement position”: it grants permission before each form, evaluates it, and catches a suspension coming back.

```
for (index, form) in forms.iter().enumerate().skip(from) {
  if permitted { grant_yield_permission(); }
  match eval(form, env.clone(), ctx) {
    Ok(value) => last = value,
    Err(EvalError::Yielded { value, mut frames }) => {
      frames.push(frame_at(index + 1)); // <- the frame is built here
      return Err(EvalError::Yielded { value, frames });
    }
    Err(other) => return Err(other),
  }
}
```

5.2.4 Step 3 — *the inner blocks run, and one of them yields*

`(let ((n 0)) ..)` builds a child environment, binds `n`, and hands its body to `eval_body_step`, which is `run_statements` for everything but the last form plus a tail call for the last. The body is one form — the `while` — so it is tail-called, and the permission is re-granted by rule 3.

The `while` arm evaluates its condition (with nothing granted — a condition is not a place to stop), finds it true, and enters `run_while`. `run_while` grants before each body form. `(log "top")` runs. `(yield n)` is reached.

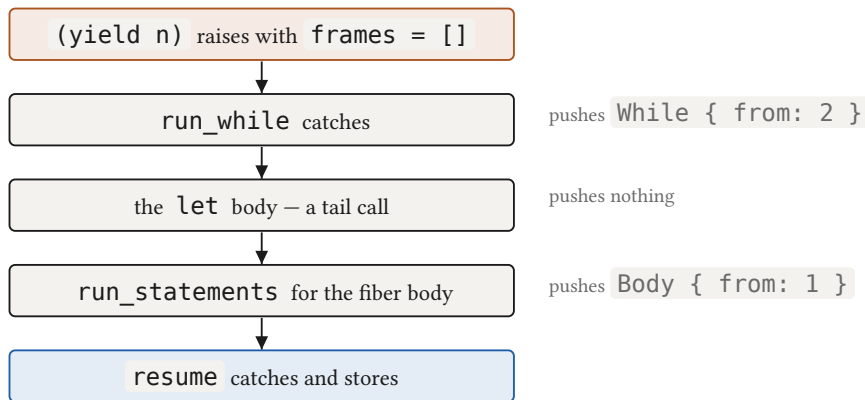
5.2.5 Step 4 — *yield raises*

```
"yield" => {
  if !permitted { return Err(EvalError::YieldNotAllowed); }
  let value = match args.first() {
    Some(form) => eval(form, env, ctx)?, // an argument: no permission
    None => LispExp::nil(),
  };
  Err(EvalError::Yielded { value, frames: Vec::new() })
}
```

The frame list starts **empty**. `yield` itself records nothing — it has nothing to record, being a leaf.

5.2.6 Step 5 — *the frames are pushed, innermost first*

The `Err` propagates outward through `?`. Each recordable block catches it, appends its own position, and re-raises:



The `let` pushes nothing because its body was in *tail* position — there is nothing after it to come back to. This is not a special case: `run_statements` only ever records for the forms before the last one, and the last one is a tail call whose own enclosing block does the recording.

The stack that reaches `resume` is therefore, innermost first:

```

pending = [
  While { condition: (< n 3), forms: [(log ..), (yield n), (setq ..)],
    from: 2, env: <the let's child env> },
  Body { forms: [(let ..)], from: 1, env: <the fiber's env> },
]

```

5.2.7 Step 6 — `resume` stores it and returns the value

```

match outcome {
  // Suspended again: the new stack is where it stopped.
  Err(EvalError::Yielded { value, frames }) => {
    fiber.pending = frames;
    Ok(value)
  }
  // Nothing left on the stack, so nothing left to come back to.
  Ok(value) => {
    fiber.is_done = true;
    Ok(value)
  }
  // Unwinding destroyed the position; see below.
  Err(other) => {
    fiber.is_done = true;
    fiber.pending.clear();
    Err(other)
  }
}

```

`(resume counter)` answers `0`, and the fiber is suspended, not done.

5.3 The ordering invariant

`frames` is **innermost first**. It is built that way because the innermost block is the first to catch the `Err` on the way out, and it is read that way because the innermost block is the first that must run on the way back in. Nothing reverses it anywhere; if a frame list is ever seen in the other order, something has reversed it by mistake.

6 Jumping back in

There is exactly one door: `resume_frames`. Everything that resumes a suspended program goes through it, including a resumption that suspends again.

```
pub fn resume_frames<T: LispContext>(frames: Vec<Frame<T>>, ctx: &T)
    -> Result<LispExp<T>, EvalError<T>>
{
    let mut last = LispExp::nil();
    let mut outer = frames.into_iter();
    while let Some(frame) = outer.next() {
        let finished = match frame {
            Frame::Body { forms, from, env } =>
                run_statements(&forms, from, &env, ctx, true, last.clone(), ..),
            Frame::While { condition, forms, from, env } =>
                run_while(&condition, &forms, from, &env, ctx, last.clone()),
        };
        match finished {
            Ok(value) => last = value,
            Err(EvalError::Yielded { value, mut frames }) => {
                frames.extend(outer);           // <- the outer blocks
                return Err(EvalError::Yielded { value, frames });
            }
            Err(other) => return Err(other),
        }
    }
    Ok(last)
}
```

Three things are going on, and each one is a bug if it is missing.

6.1 Innermost first, one frame at a time

Each frame is run from its own `from` index. A `Body` frame runs its remaining forms; a `While` frame finishes the interrupted iteration and then loops as if nothing had happened — which is why `run_while` is the same function the `while` special form itself uses, entered at a different index. An interrupted loop and an uninterrupted one cannot drift apart, because there is only one loop.

6.2 The outer frames are carried forward

If resuming a frame suspends again — which is exactly what a loop that yields every iteration does — the frames *outside* it have not been reached yet. They are still the way back, so `frames.extend(outer)` appends them to the new suspension.

Without this line, a fiber written as a loop escapes its own loop the second time it yields: the first resumption pushes a fresh `While` frame, the enclosing `Body` frame is dropped on the floor, and the third `resume` finds a program that has mysteriously forgotten it was inside anything. The failure is invisible on a fiber that yields once, which is what makes it worth a test of its own.

6.3 The value is carried outward

`run_statements` takes a `carried` value and starts its `last` from it, rather than from `nil`.

A block whose last form was the one that yielded has nothing left to run when it is resumed — and it is worth whatever the blocks *inside* it came to, not `nil`. Consider:

```
(fiber (progn (yield) 7))
```

The second `resume` runs the inner `Body` frame, which evaluates `7`. Then it reaches the outer `Body` frame, which has nothing left. Starting that frame's `last` at `nil` would write `nil` over the `7` that had just been computed, and the fiber would answer `nil`. Carrying the value through gives `7`.

7 Interaction with everything else

7.1 Errors and throws

A `yield` is not an error and is not caught by anything that catches errors. It does not need to be *excluded* from `condition-case` either — a suspension cannot originate inside one, because permission is never granted there. The exclusion is structural, not a check.

The reverse direction is a decision rather than a consequence: an error inside a fiber **ends** it.

```
Err(other) => { fiber.is_done = true; fiber.pending.clear(); Err(other) }
```

Unwinding destroyed the position the fiber was at. Keeping `pending` would mean a later `resume` restarting the program from a place it had already partly run, with side effects already done. Ending it is the only answer that does not silently repeat work.

7.2 Fuel

Fuel is thread-local and counted per evaluation step. It interacts with fibers in three ways, all of them by not interacting:

- A suspension costs no fuel and refunds none. It is a return, not a loop.
- A `resume` runs on *the caller's* budget. A host that wants a fiber bounded independently of whatever called it sets the thread's remaining fuel before resuming — which is what this interpreter's editor does for its background workers, giving each turn its own allowance so that a fiber which never reaches a `yield` cannot hold the thread it shares.
- Running out of fuel is an ordinary error, so by the rule above it ends the fiber. That is deliberate: a fiber that exhausted a budget mid-form has no usable position to come back to.

7.3 Threads

`Env` is `Send + Sync`, and a `SharedFiber` is an `Arc<RwLock<.,.>>`, so a fiber may be created on one thread and resumed on another. Two properties make that safe:

1. `resume` takes the frame stack out from under the lock and releases the lock before evaluating. Arbitrary Lisp never runs with the fiber's lock held.
2. Because the stack is taken rather than borrowed, a concurrent `resume` finds an empty stack and returns `nil`. Two threads cannot run one fiber at once.

What is *not* provided is protection for whatever the fiber touches. Two fibers sharing a variable share it exactly as two functions would.

8 Writing an event loop

Everything above is the mechanism. This is the part a host writes.

A fiber does not schedule itself. It is a program that can be paused, and nothing more — when it runs, for how long, and what happens to the value it yields are all the host’s business. That separation is why the interpreter knows nothing about timers, sockets or frames.

8.1 The minimum loop

A scheduler needs three operations, and the interpreter provides all three through ordinary Lisp values:

Operation	How
Hold a fiber	keep the <code>LispExp::Fiber</code> . It is <code>Clone</code> , and clones share one program — resuming either advances both, because they are the same <code>Arc</code> .
Advance it one turn	evaluate <code>(resume f)</code> , or call <code>resume_frames</code> on a stack taken from its state.
Ask whether it is finished	<code>(fiber-done-p f)</code> from Lisp, or <code>LispExp::fiber_is_done()</code> from Rust, which answers <code>None</code> for anything that is not a fiber.

In pseudocode, and this really is the whole of it:

```
loop {
  wait_for_something();           // a timer, a socket, a frame
  for fiber in runnable {
    let value = resume(fiber);    // one turn
    deliver(value);              // whatever yielding means to you
    if fiber.is_done() { retire(fiber); }
  }
}
```

8.2 What the yielded value is for

`(yield V)` hands `V` to whoever called `resume`, and the host decides what that means. It is the only channel from a fiber to its scheduler, and three conventions cover nearly everything:

A progress report the fiber yields how far it has got, and the host uses it to draw a bar or decide whether to keep going.

A request the fiber yields a description of something it needs — bytes from a socket, a file to be read — and the host performs it, leaving the answer somewhere the fiber will look when it is resumed.

Nothing at all `(yield)` means only “**I have done a sensible amount of work; come back when you like**”. This is the common case for a worker that is simply being polite about a shared thread.

There is no channel in the other direction built into `resume`. A host that needs one puts the answer in the environment the fiber can see, or in a shared structure, before resuming.

8.3 Worked example: a cooperative scheduler

A host that wants several fibers to make progress fairly, with no threads, needs nothing beyond a list:

```
(setq scheduler--tasks nil)

(defun spawn-task (f)
```

```

"Add fiber F to the round-robin."
(setq scheduler--tasks (append scheduler--tasks (list f)))

(defun scheduler-tick ()
  "Give every live task one turn. Returns how many are still running."
  (let ((live nil))
    (mapc (lambda (f)
            (resume f)
            (if (fiber-done-p f) nil (setq live (append live (list f)))))
          scheduler--tasks)
    (setq scheduler--tasks live)
    (length live)))

```

Each task is written as a loop that yields where it is willing to be interrupted:

```

(spawn-task
 (fiber
  (let ((n 0))
    (while (< n 100)
      (do-one-chunk-of-work n)
      (setq n (+ n 1))
      (yield)))))

```

`scheduler-tick` is then called from wherever the host's own loop already is — a timer, an idle callback, the top of a frame — and the tasks interleave. Nothing in the interpreter had to know that a scheduler existed.

This example is not illustrative. It is evaluated by `the_documented_cooperative_scheduler_runs` in `lisp/tests/yield_tests.rs`, so if it stops being true of the interpreter the test suite says so.

8.4 Worked example: this interpreter's editor

The editor that embeds this interpreter drives fibers from a thread that also runs syntax colouring and other periodic work. It is a useful example because it shows the three decisions a real host has to make, none of which the interpreter takes for it.

When to resume on a timer, at the same interval as syntax colouring. The host chooses the interval; a host driven by a socket would resume when the socket became readable instead.

How much a turn may do each turn sets the thread's remaining fuel to a private allowance before resuming, so a fiber that never reaches a `yield` ends its own turn rather than holding the shared thread. This is the interpreter's `set_remaining`, used as host policy.

What to do when a turn ends badly an error, or a spent allowance, retires the fiber and writes one diagnostic. A host that retried instead would run a broken fiber for the life of the process.

In Lisp, the whole of it looks like this:

```

(define-worker 'indexer
  (fiber
   (while t
     (if (index--stale-p)
         (index--one-chunk)
         nil)
     (yield)))))

```

and a fiber that finishes retires itself, because `fiber_is_done` says so.

8.5 What a fiber may not do

Nothing in the interpreter restricts what a suspended program may call — it is the same Lisp it always was. What the *host* restricts is its own business, and a host driving fibers off the main thread usually has at least one rule of this kind. The editor's is that a fiber may not open a prompt: a prompt put up by a background turn is a window nobody is typing into, over a command nobody started.

The interpreter has no opinion about that. A host enforces such a rule by asking, in its own primitives, whether it is inside a turn.

9 Reference

9.1 The Lisp surface

Form	Meaning
<code>(fiber FORM...)</code>	A suspendable program. The body is a <code>progn</code> .
<code>(yield)</code> / <code>(yield V)</code>	Stop here; hand <code>V</code> (or <code>nil</code>) to <code>resume</code> . Legal only in statement position.
<code>(resume F)</code>	Carry <code>F</code> on from where it stopped. Answers the value it stopped with, or <code>nil</code> once it is finished.
<code>(fiber-done-p F)</code>	<code>t</code> when <code>F</code> has nothing left to run. The only way to tell a finished fiber from a parked one — <code>resume</code> answering <code>nil</code> cannot, since a fiber may perfectly well yield <code>nil</code> .

9.2 The Rust surface

Item	Role
<code>Frame</code>	One block with work left in it.
<code>FiberState</code>	<code>pending: Vec<Frame></code> and <code>is_done</code> .
<code>grant_yield_permission()</code>	Allow the next single evaluation on this thread to suspend. A host calls it only if it drives frames directly.
<code>resume_frames(frames, ctx)</code>	Run a frame stack to completion or to the next suspension.
<code>LispExp::fiber_is_done()</code>	<code>Some(bool)</code> for a fiber, <code>None</code> otherwise. <code>fiber-done-p</code> is this, exposed to Lisp.
<code>EvalError::Yielded { value, frames }</code>	A suspension in flight.
<code>EvalError::YieldNotAllowed</code>	A <code>yield</code> somewhere no frame could be recorded.

9.3 Invariants worth not breaking

1. `frames` is innermost first, everywhere, always.
2. Permission defaults to false; only positions that can record a frame grant it; every evaluation consumes it.
3. `resume` takes the frame stack out from under the lock and never evaluates with the lock held.
4. A resumption that suspends again re-appends the outer frames it had not reached.
5. A block with nothing left to run is worth what the blocks inside it came to, not `nil`.
6. An error inside a fiber ends it.

9.4 Where the code is

File	What is in it
<code>lisp/types.rs</code>	<code>Frame</code> , <code>FiberState</code> .
<code>lisp/error.rs</code>	<code>Yielded</code> , <code>YieldNotAllowed</code> .
<code>lisp/eval.rs</code>	The permission flag, <code>run_statements</code> , <code>eval_body_step</code> , <code>run_while</code> , <code>resume_frames</code> , the <code>yield</code> and <code>fiber</code> special forms.
<code>lisp/base/fibers.rs</code>	<code>resume</code> .